

Data Communications

Lecture #7

What we have discussed

- Video Streaming / Content Distribution Networks (CDN)

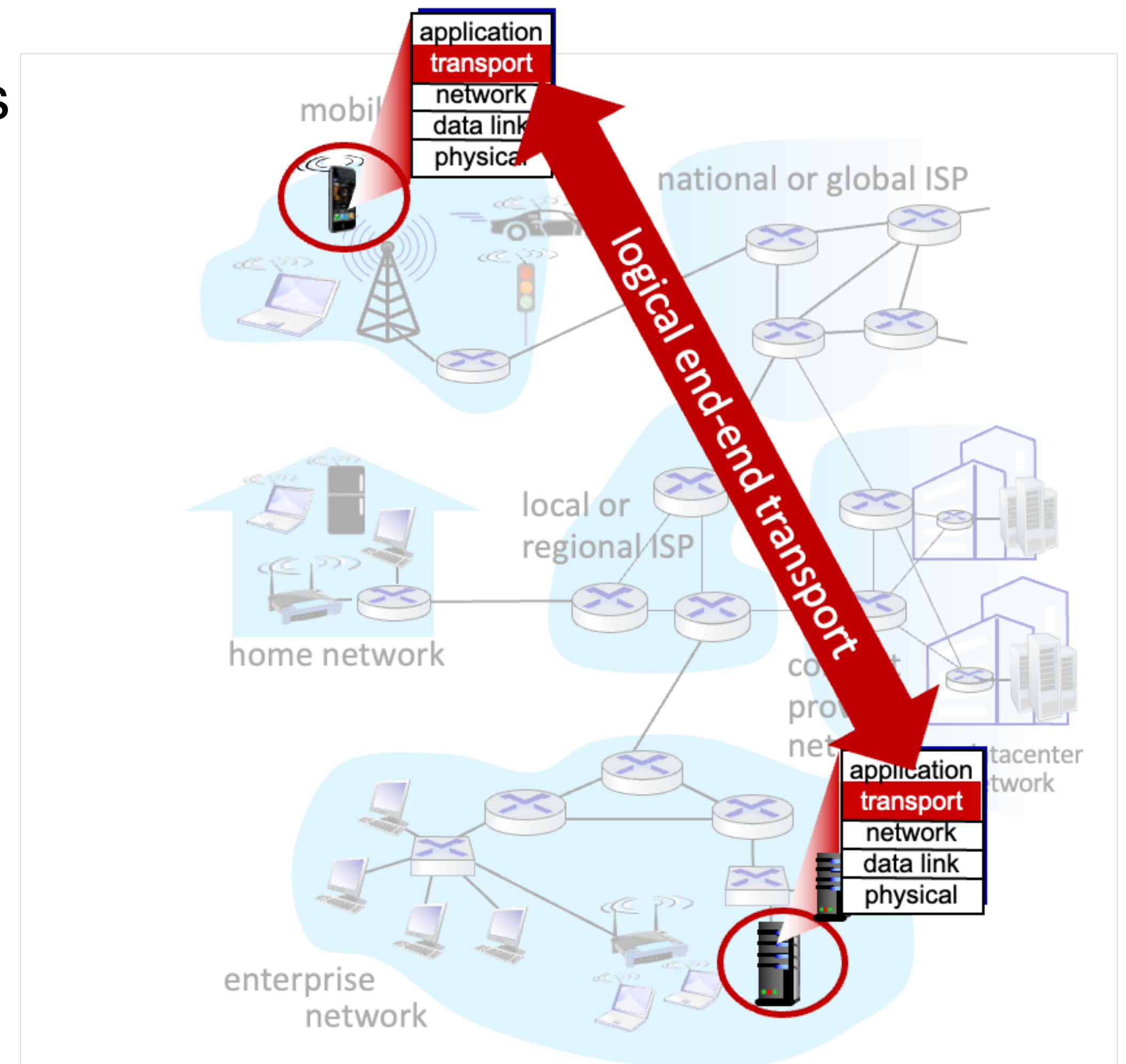
Today...

- Transport layer services
- Multiplexing / Demultiplexing
- Connectionless transport: UDP

Transport layer services

Transport layer services and protocols

- Provide logical communication between **application processes** running on different hosts
- Actions of transport protocols in end system
 - Sender: breaks application messages into **segments**, passes to network layer
 - Receivers: reassembles segments into messages, passes to application layer
- Two transport protocols available to Internet applications
 - TCP, UDP



Transport vs. Network layer services and protocols



- Household analogy: 12 kids in Ann's house sending letters to 12 kids in Bill's house
 - Houses: Hosts
 - Kids: Processes
 - Letters in envelopes: App messages
 - Ann / Bill who demux to in-house sibling: Transport layer protocols
 - Postal service: Network layer protocol

Transport vs. Network layer services and protocols

- **Transport layer**

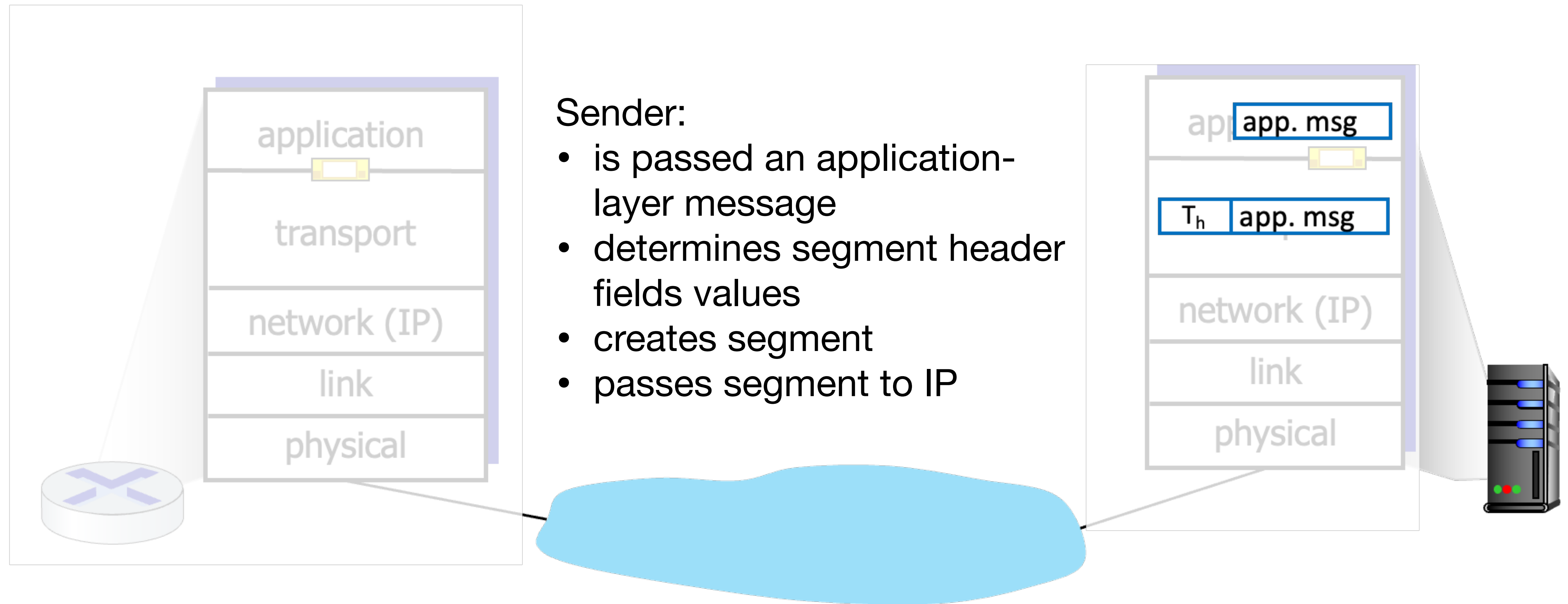
- Logical communication between processes
- Relies on, enhances, network layer services

- **Network layer**

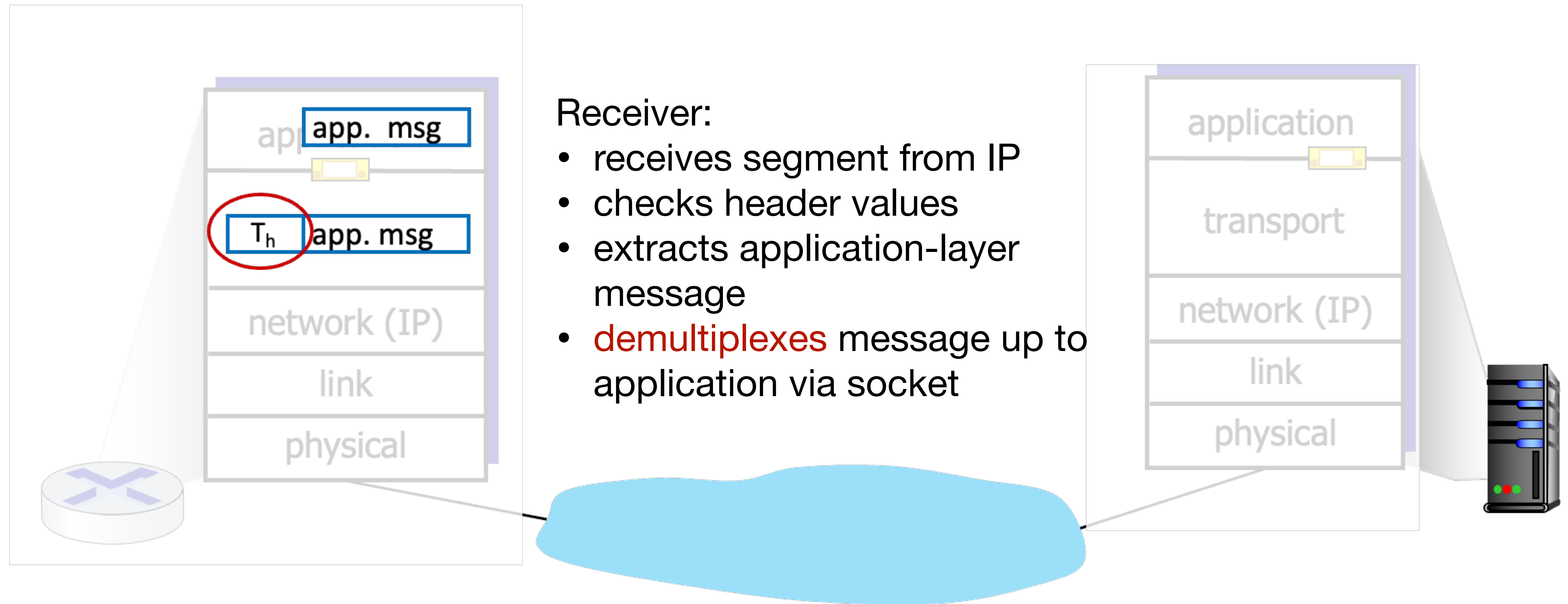
- Logical communication between hosts

- Household analogy: 12 kids in Ann's house sending letters to 12 kids in Bill's house
 - Houses: Hosts
 - Kids: Processes
 - Letters in envelopes: App messages
 - Ann / Bill who demux to in-house sibling: Transport layer protocols
 - Postal service: Network layer protocol

Transport layer actions

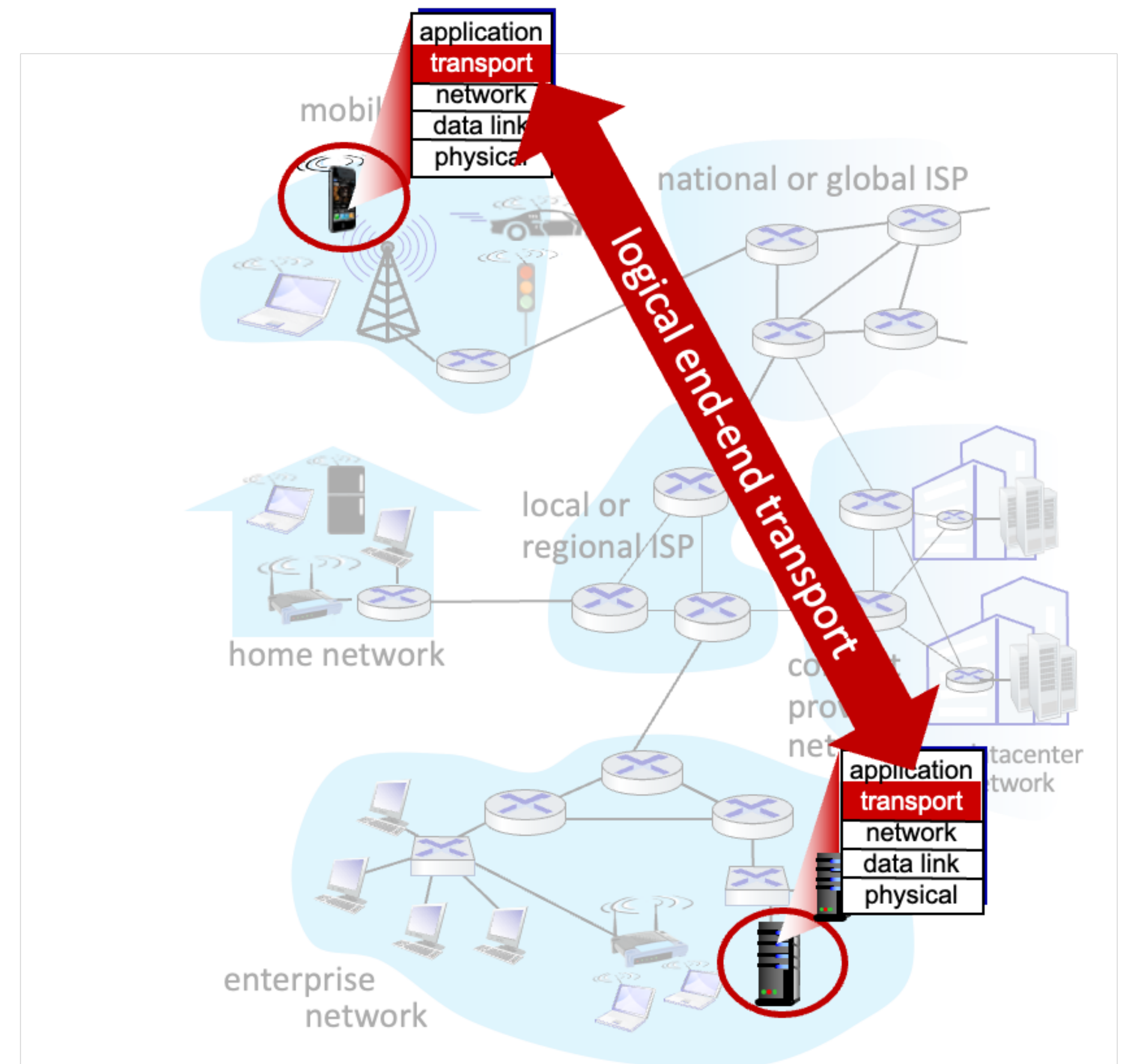


Transport layer actions



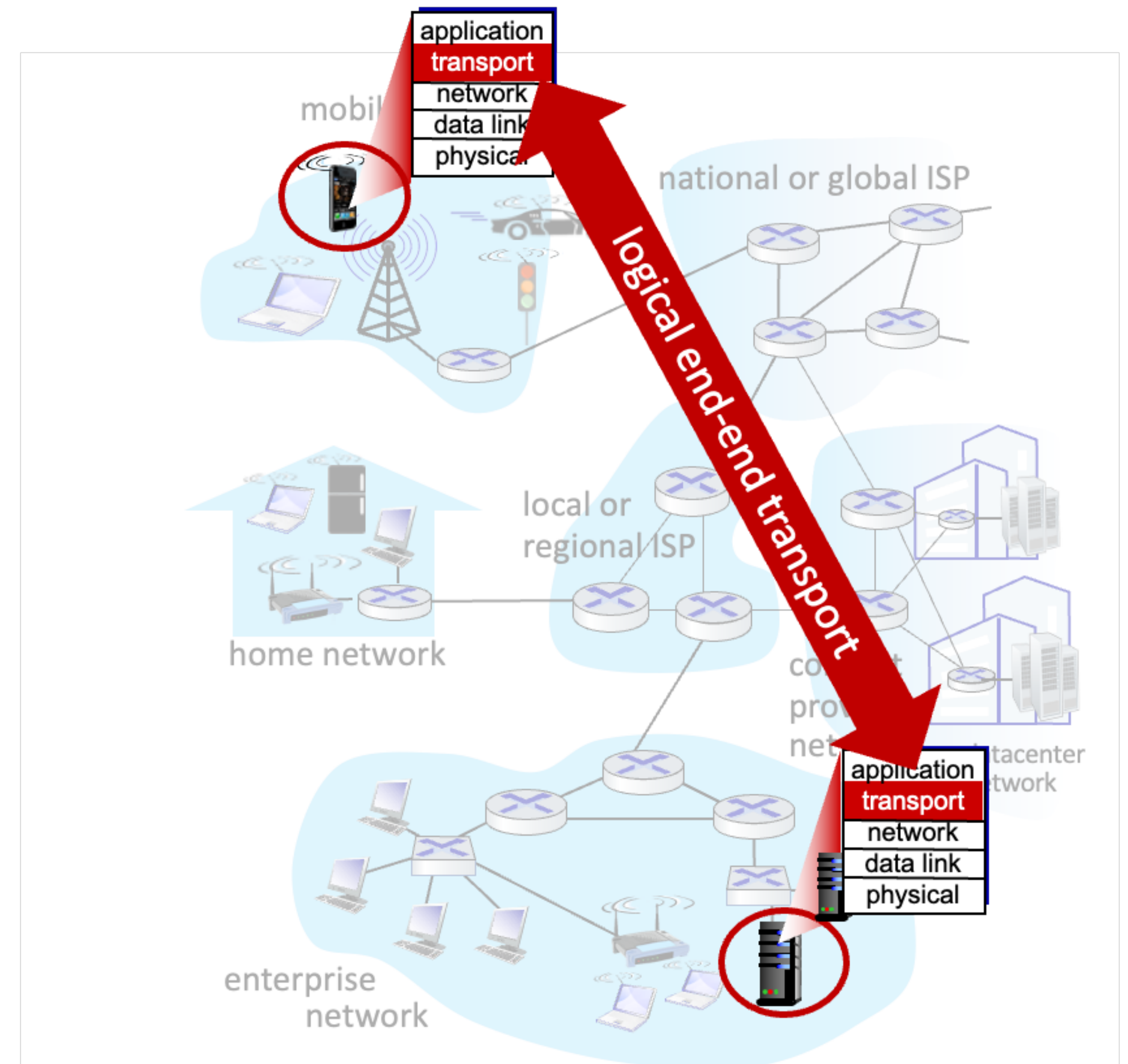
Two principal Internet transport protocols

- TCP: Transmission Control Protocol
 - Reliable, in-order delivery
 - Congestion control
 - Flow control
 - Connection setup
- UDP: User Datagram Protocol
 - Unreliable, unordered delivery
 - No-frills extension of “best-effort” IP



Two principal Internet transport protocols

- Services not available
 - Delay guarantee
 - Bandwidth guarantee

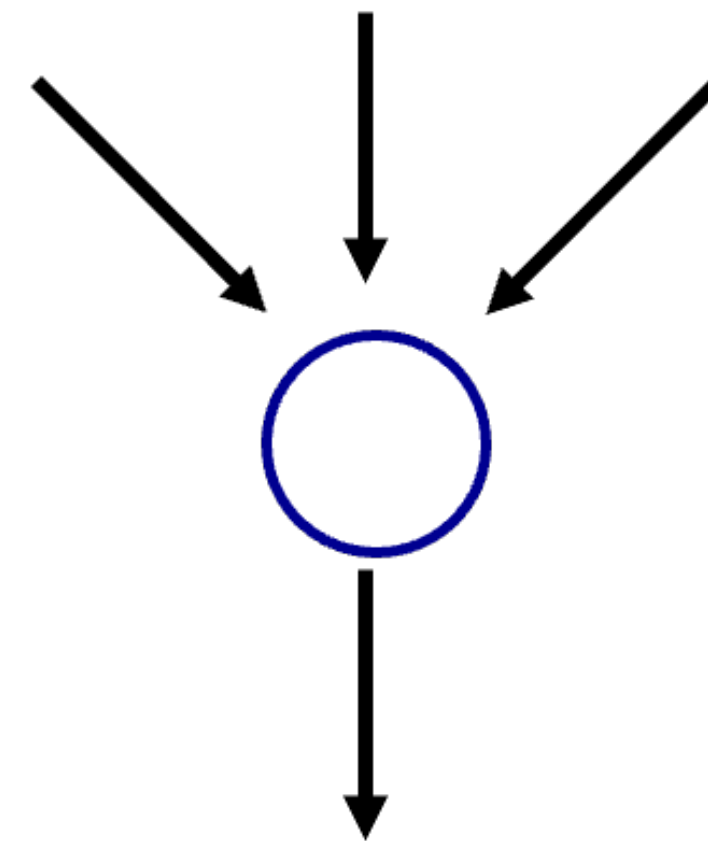


Multiplexing & Demultiplexing



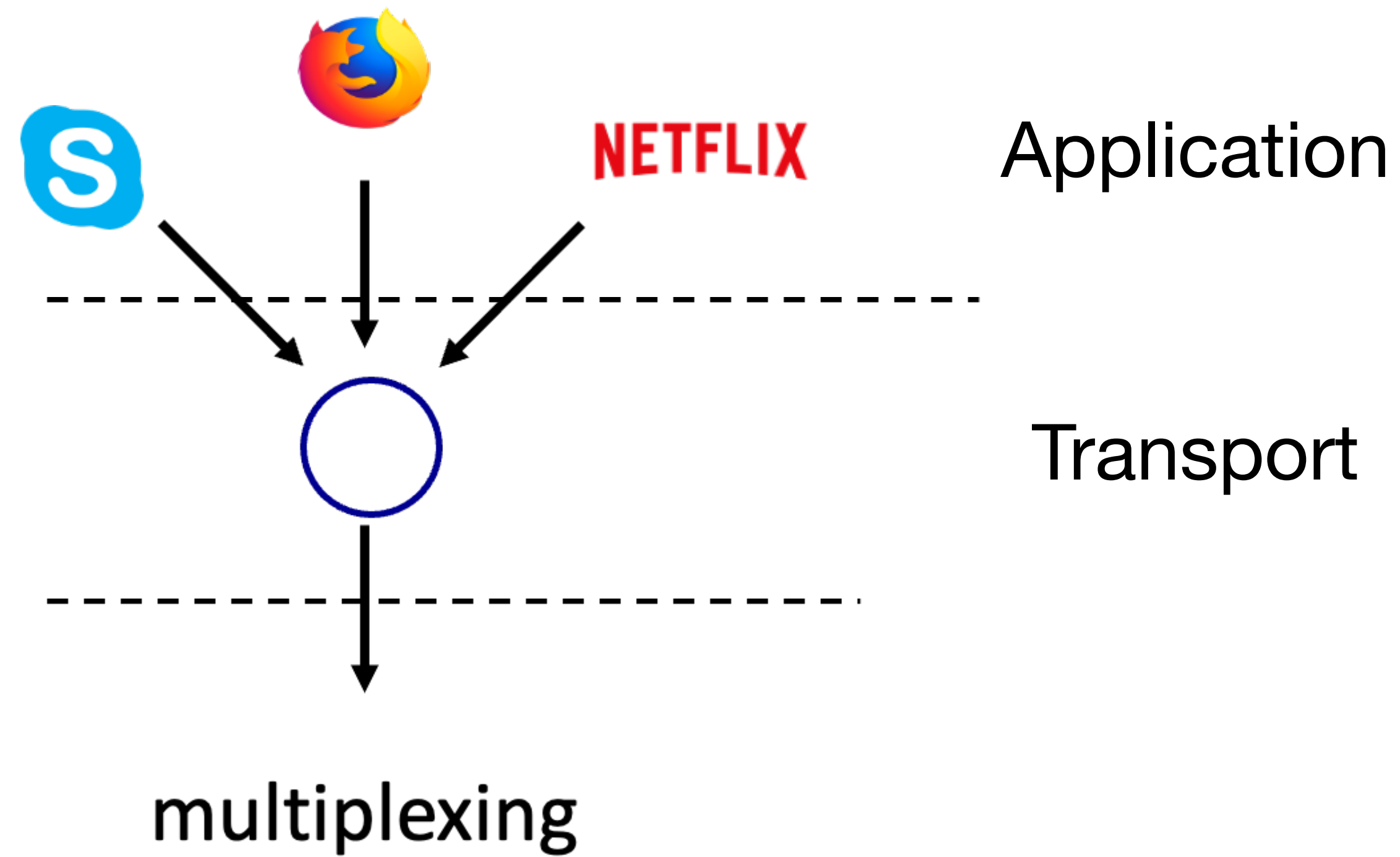
Multiplexing

Multiplexing



multiplexing

Multiplexing

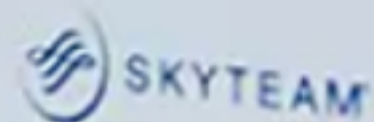




Demultiplexing

AIRFRANCE 

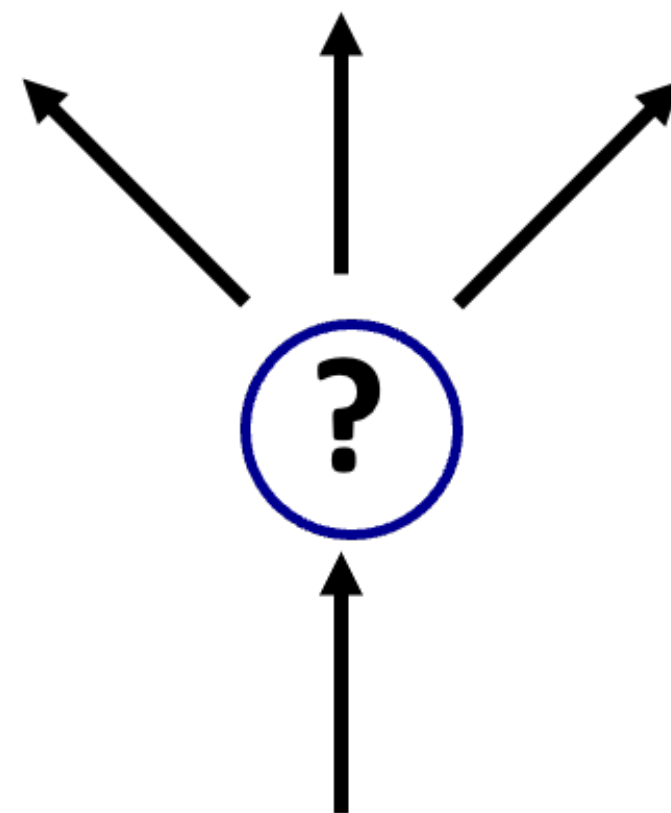
ECONOMY 



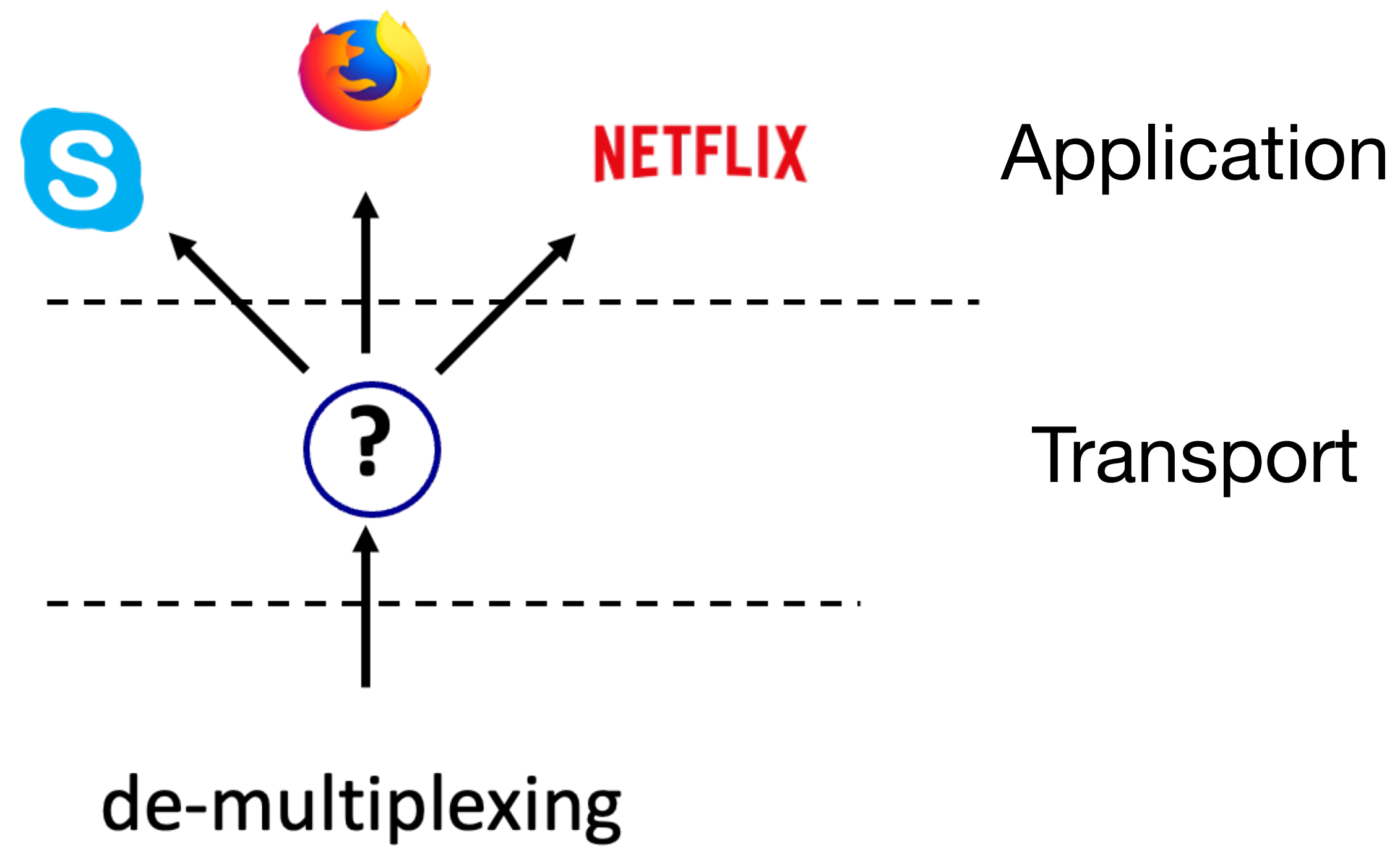
AIRFRANCE 

SKY
PRIORITY™





de-multiplexing



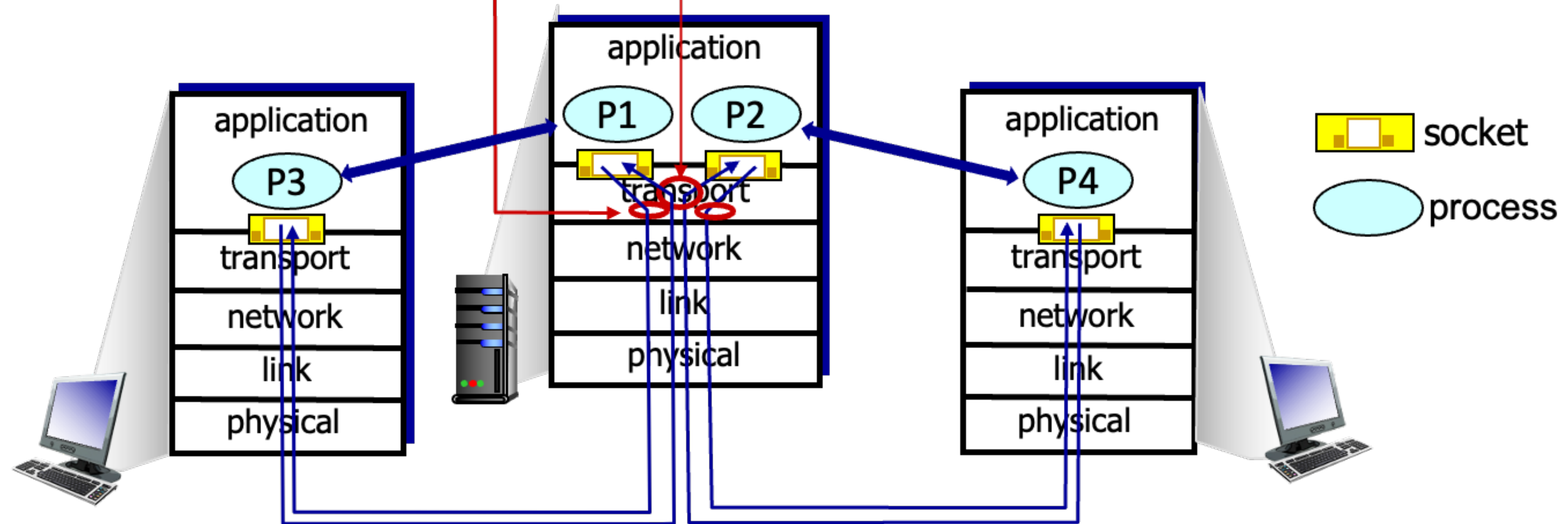
Multiplexing / demultiplexing

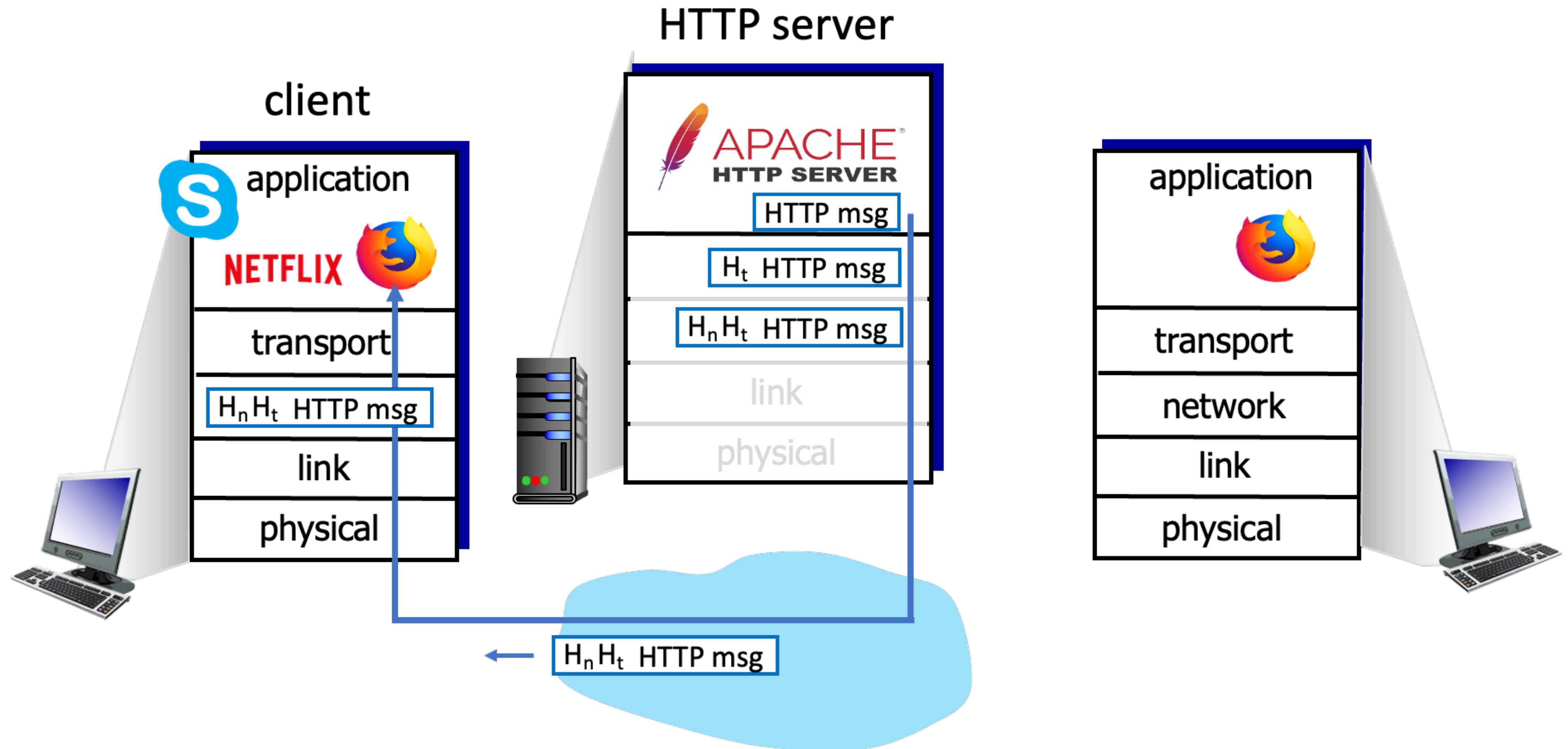
multiplexing as sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing as receiver:

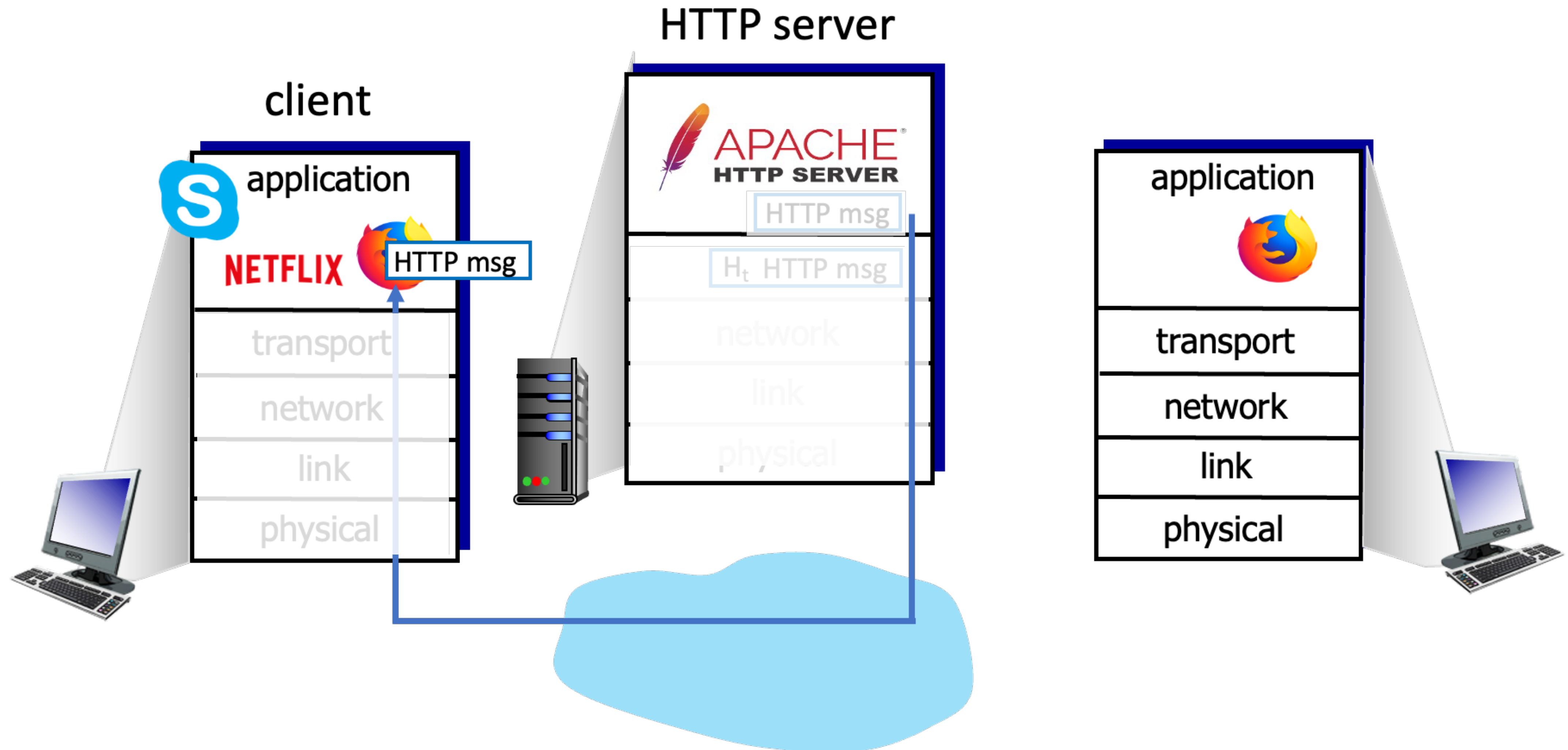
use header info to deliver received segments to correct socket





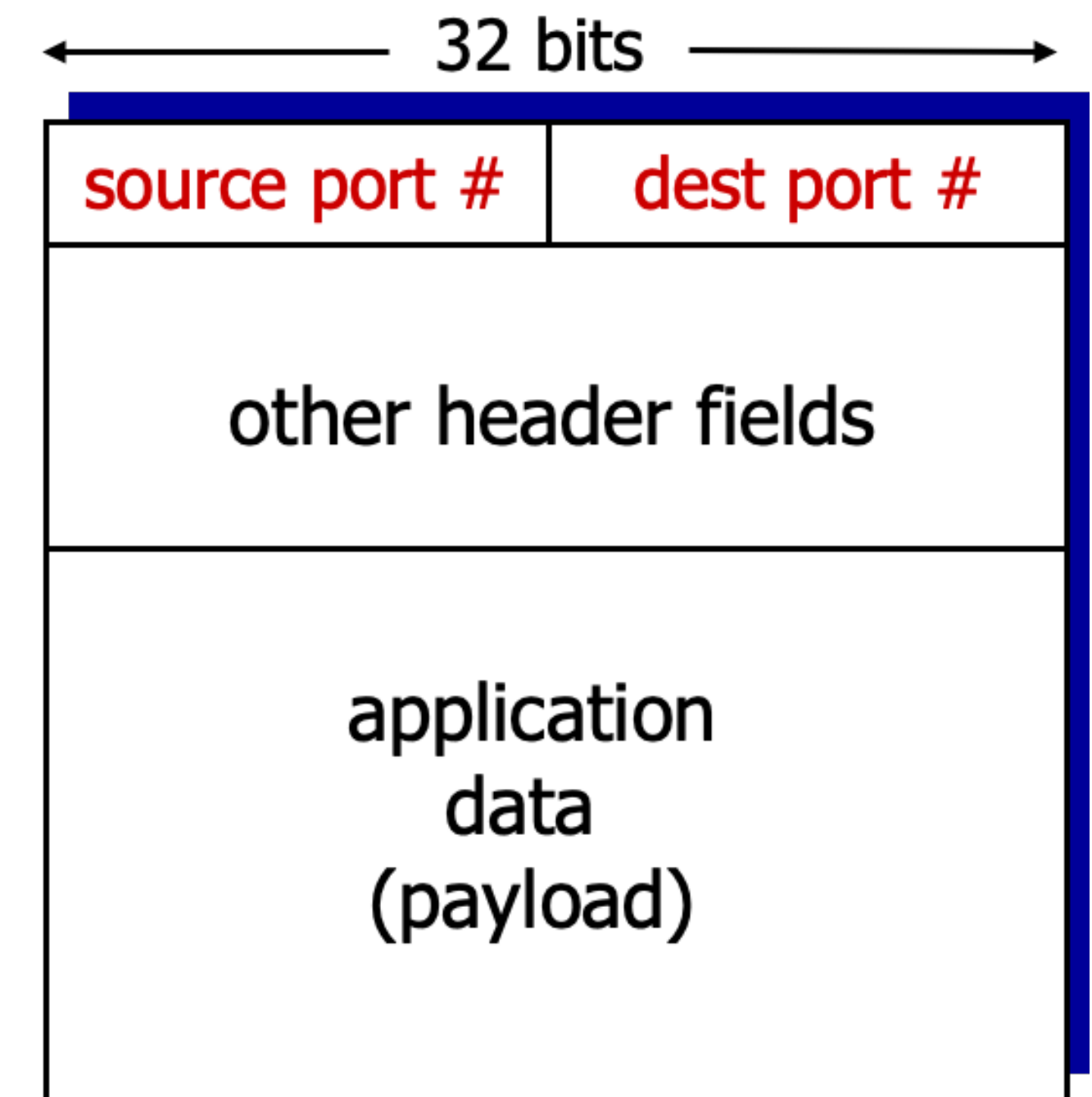


How does transport layer know to deliver message to Firefox browser process rather than Netflix process or Skype process?



How demultiplexing works

- Host receives IP datagrams
 - Each datagram has source IP address, destination IP address
 - Each datagram carries one transport-layer segment
 - Each segment has source port number, destination port number
- Host uses **IP addresses & port numbers** to direct segment to appropriate socket



TCP/UDP segment format

Connectionless demultiplexing

- Recall

- When creating socket, **host-local port #** must be specified

```
DatagramSocket mySocket = new DatagramSocket (12345)
```

- When creating datagram to send into UDP socket the following must be specified

- **Destination IP address**

- **Destination port #**

- When receiving UDP segment, a host does

- Check destination port # in segment
 - Direct UDP segment to socket with that port #

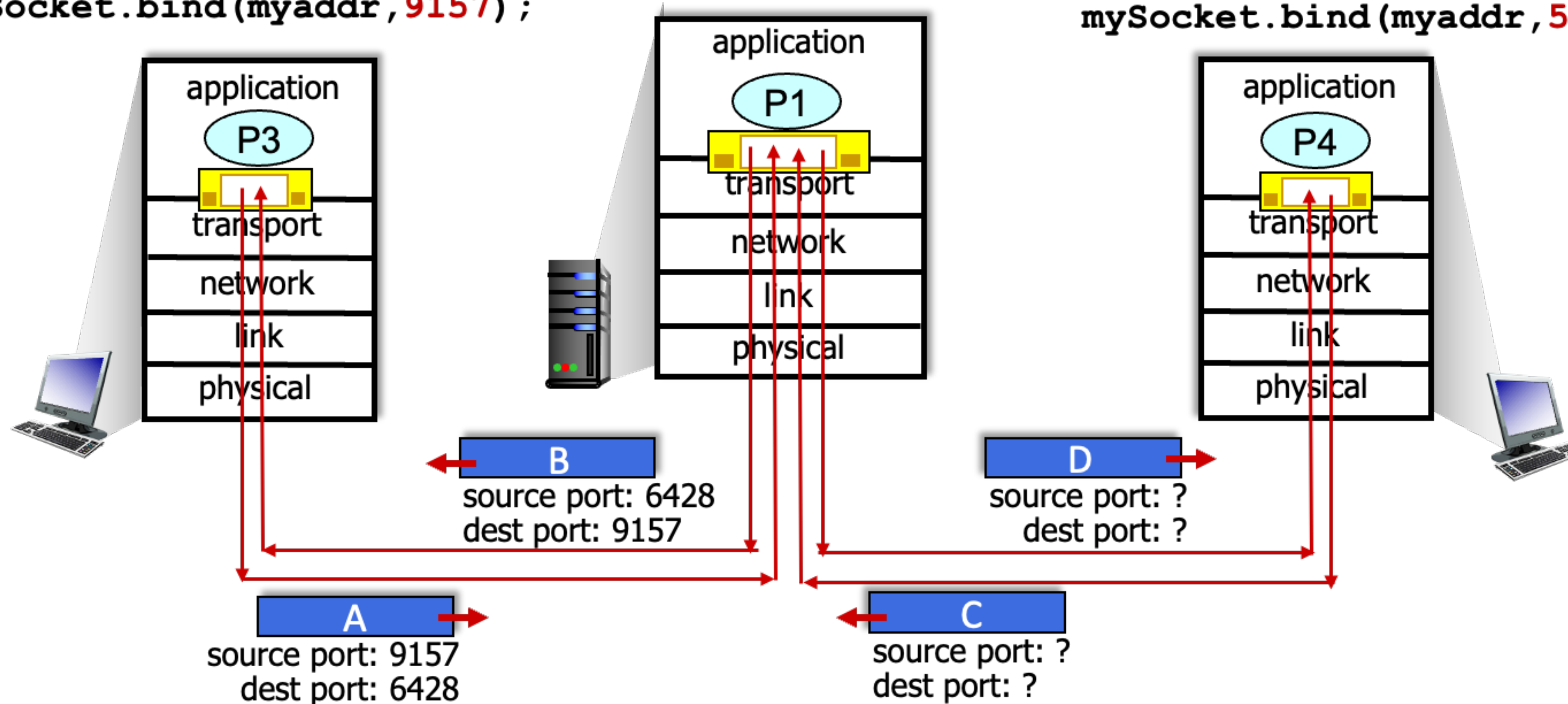
⇒ IP datagram with **the same destination port #**, but different source IP addresses and/or source port # will be directed to the **same socket** at receiving host

Connectionless demultiplexing example

```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 6428);
```

```
mySocket =  
    socket(AF_INET, SOCK_STREAM)  
mySocket.bind(myaddr, 9157);
```

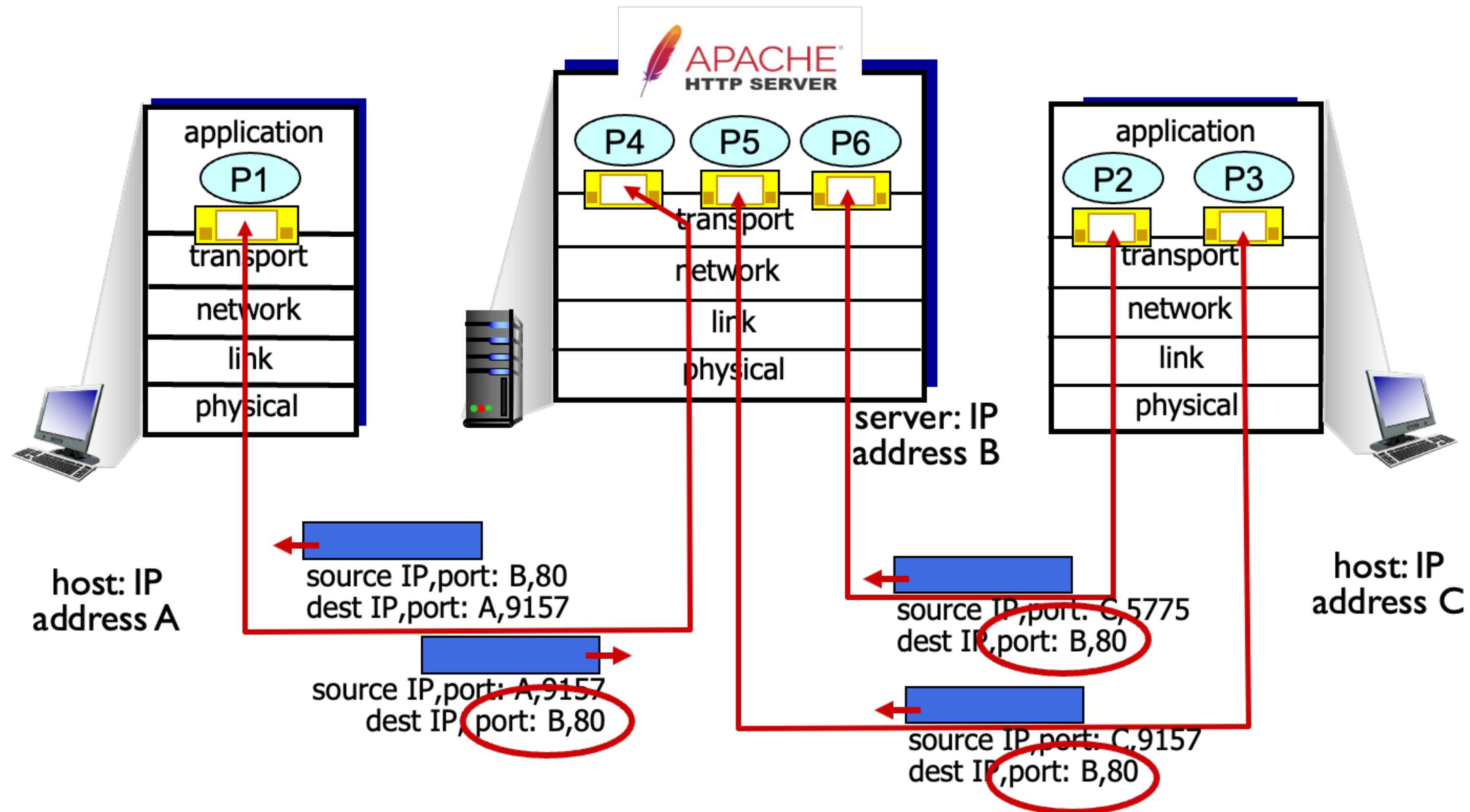
```
mySocket =  
    socket(AF_INET, SOCK_STREAM)  
mySocket.bind(myaddr, 5775);
```



Connection-oriented demultiplexing

- TCP socket is identified by 4-tuple
 - ⇒ (Source IP address, Source port #, Destination IP address, Destination port #)
- Demux: receiver uses **all four values** to direct segment to appropriate socket
- Server host may support many simultaneous TCP sockets
 - Each socket is identified by its own 4-tuple
 - Each socket is associated with a different connecting client

Connectionless demultiplexing example



- ✓ All three segments are destined to IP address of B.
- ✓ Destination port # (80) are demultiplexed to different sockets.

Connectionless transport: UDP

UDP: User Datagram Protocol

- “No-frills”, “bare-bone” Internet transport protocol
- “Best-effort” service
 - UDP segment may be lost, but no further actions will be done for the loss
 - Delivered out-of-order to apps
- Connectionless
 - No handshaking between sender and receiver
 - Each UDP segment is handled independently of others.

UDP: User Datagram Protocol

- Why is there a UDP?
 - No connection establishment (which can contribute a couple of RTT to the delay)
 - Simple: no connection state at sender and receiver
 - Small header size
 - No congestion control
 - UDP can blast away as fast as desired
 - Can function in the face of congestion

UDP: User Datagram Protocol

- UDP use
 - Streaming multimedia apps (lost tolerant, rate sensitive)
 - DNS, SNMP,
 - HTTP/3, ...
- If reliable transfer is needed over UDP (e.g., HTTP/3)
 - Add needed reliability at the application layer
 - Add congestion control at the application layer

UDP: User Datagram Protocol (RFC 768)

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

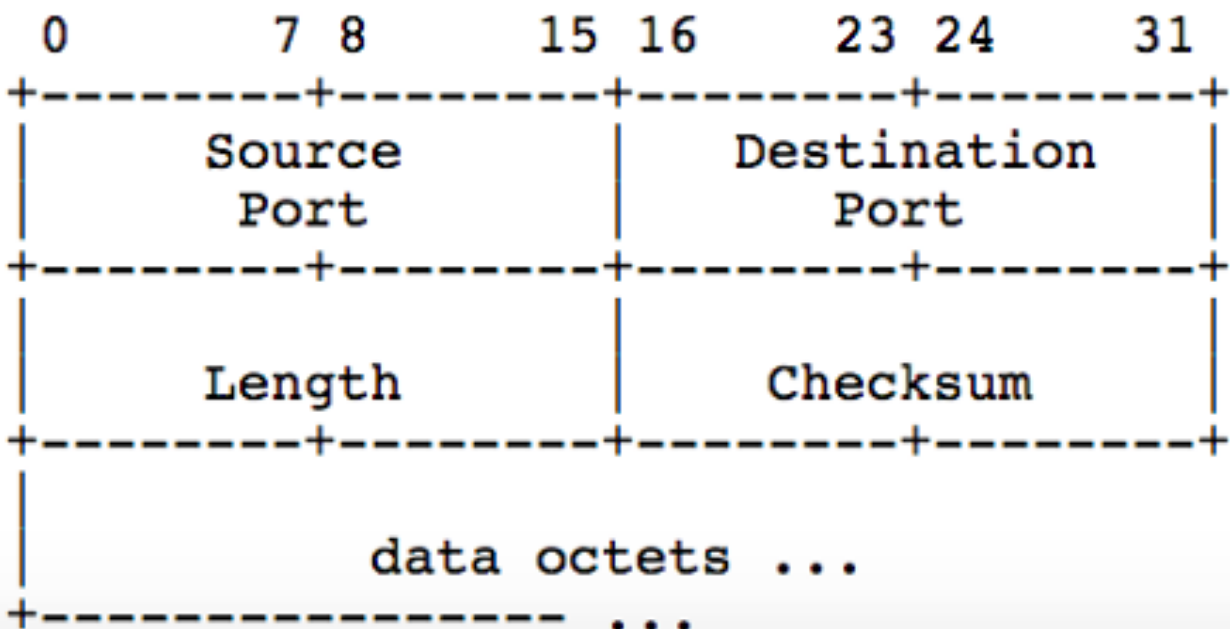
User Datagram Protocol

Introduction

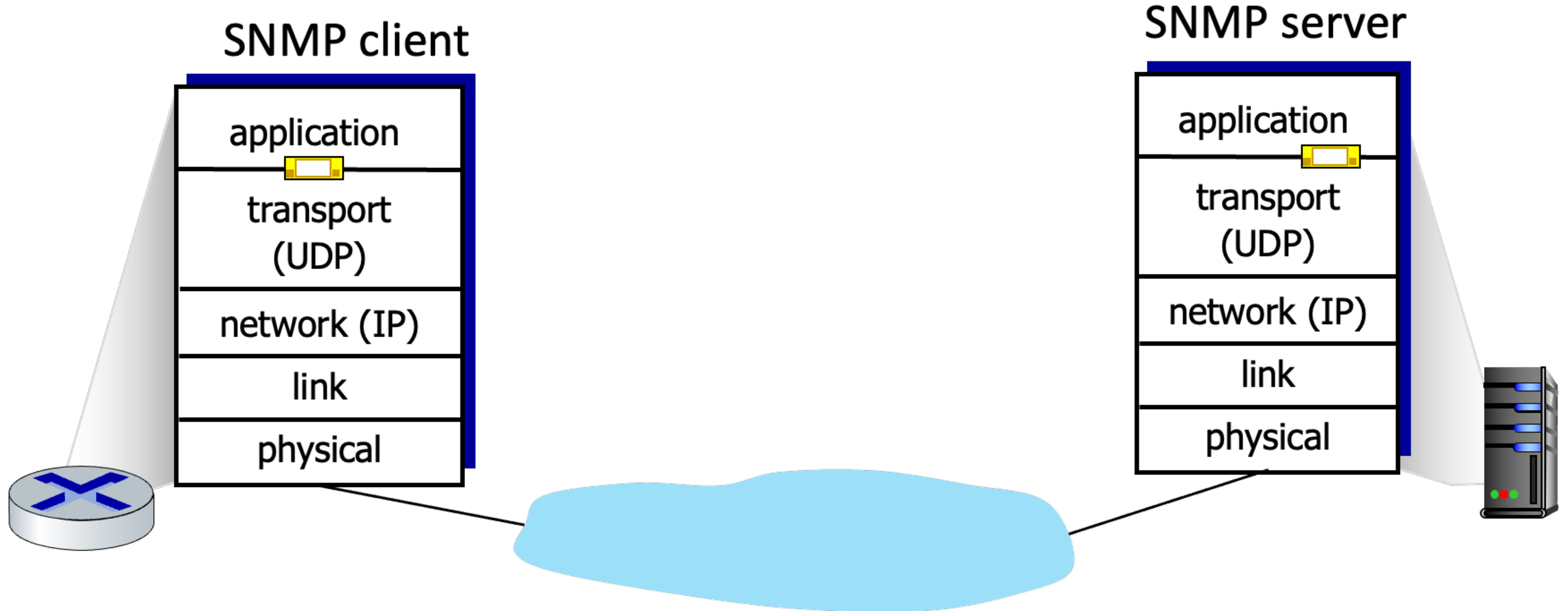
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

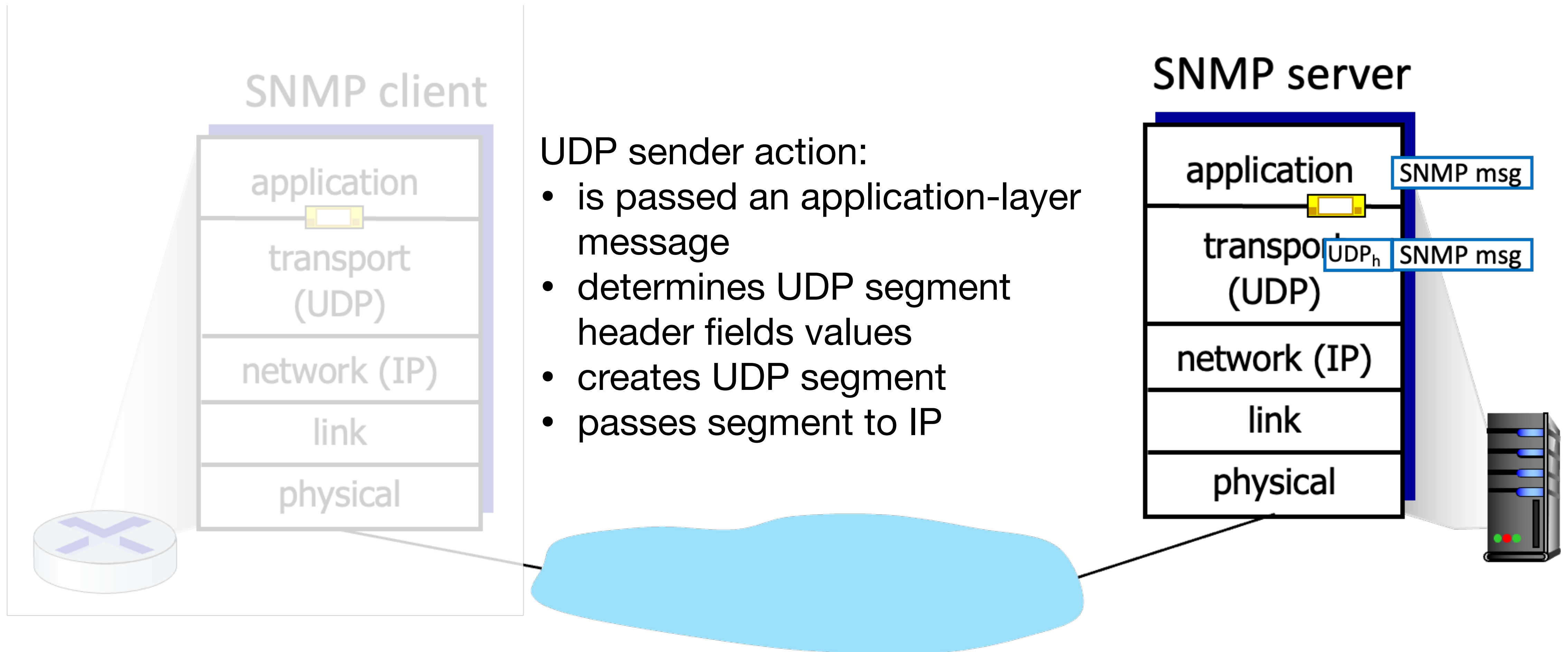
Format



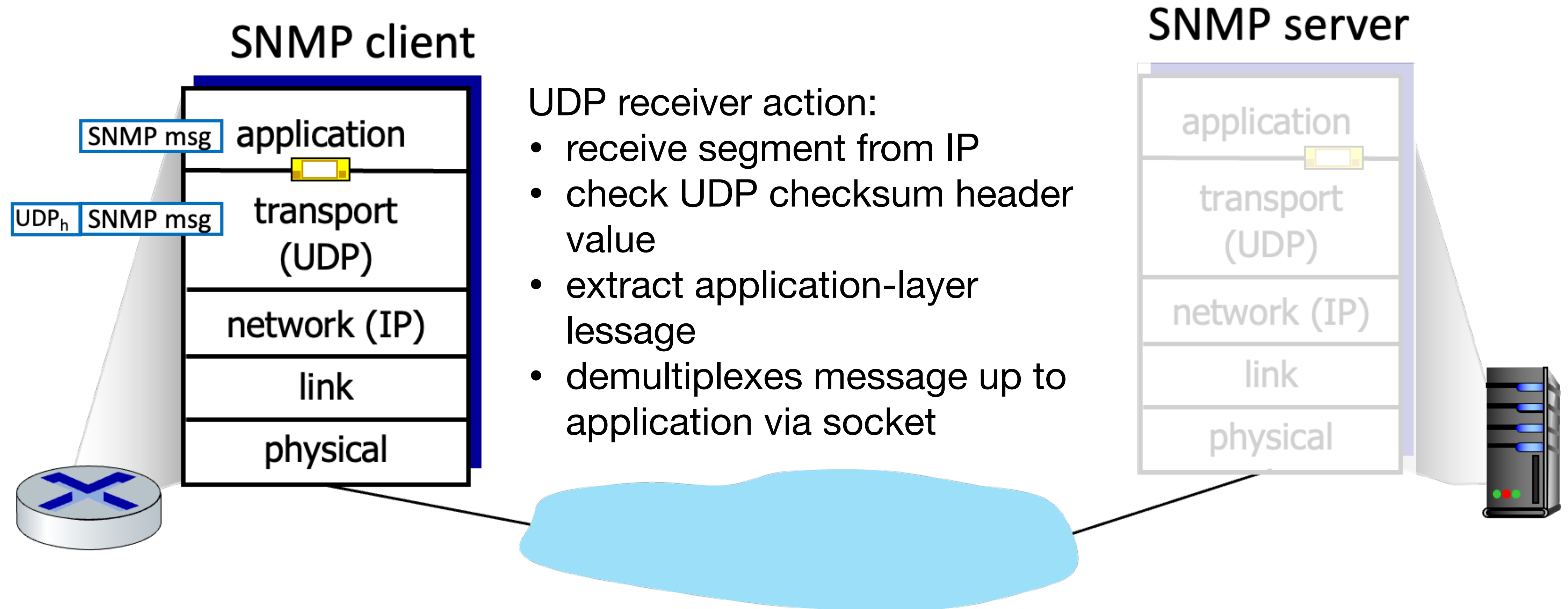
UDP: transport layer actions



UDP: transport layer actions

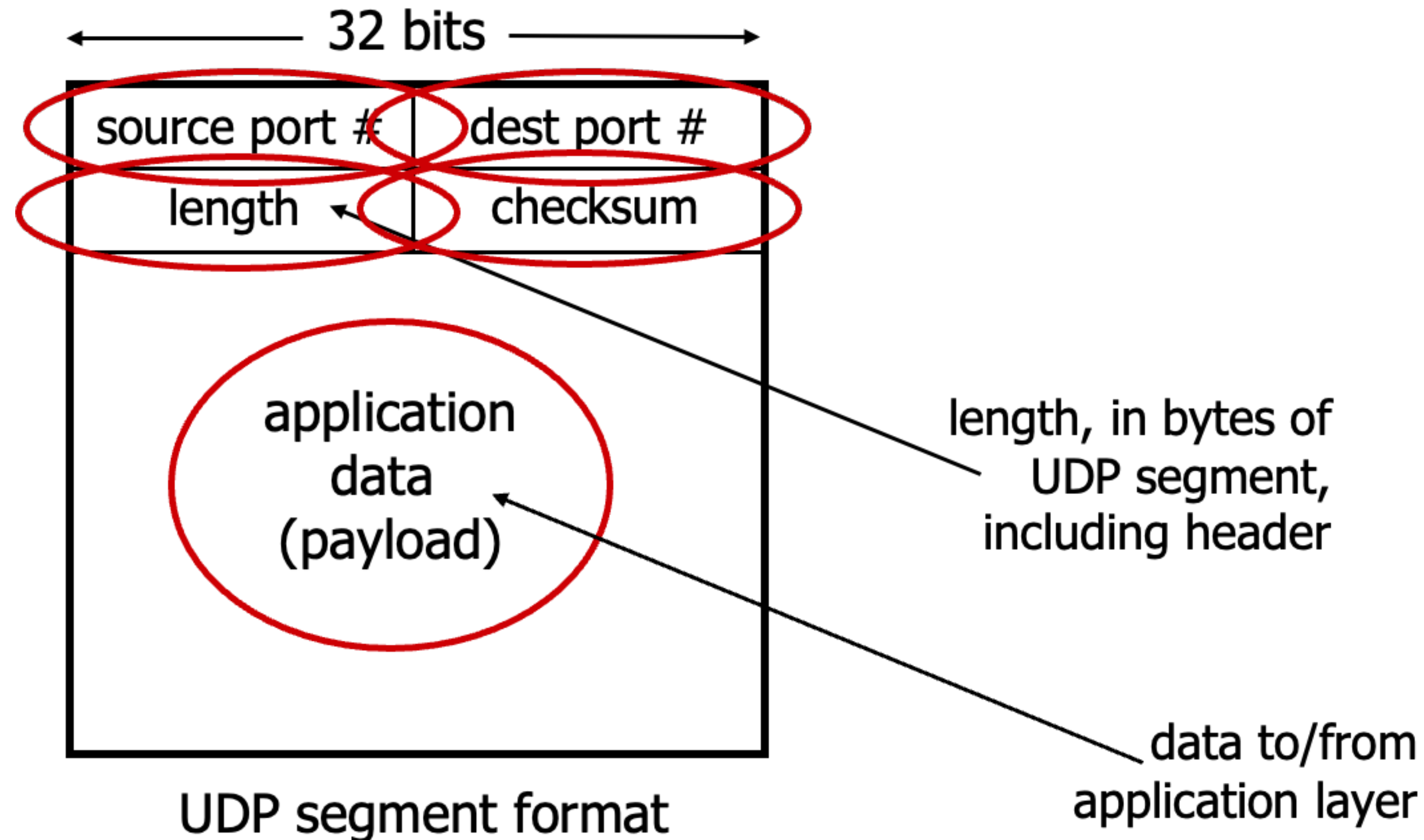


UDP: transport layer actions



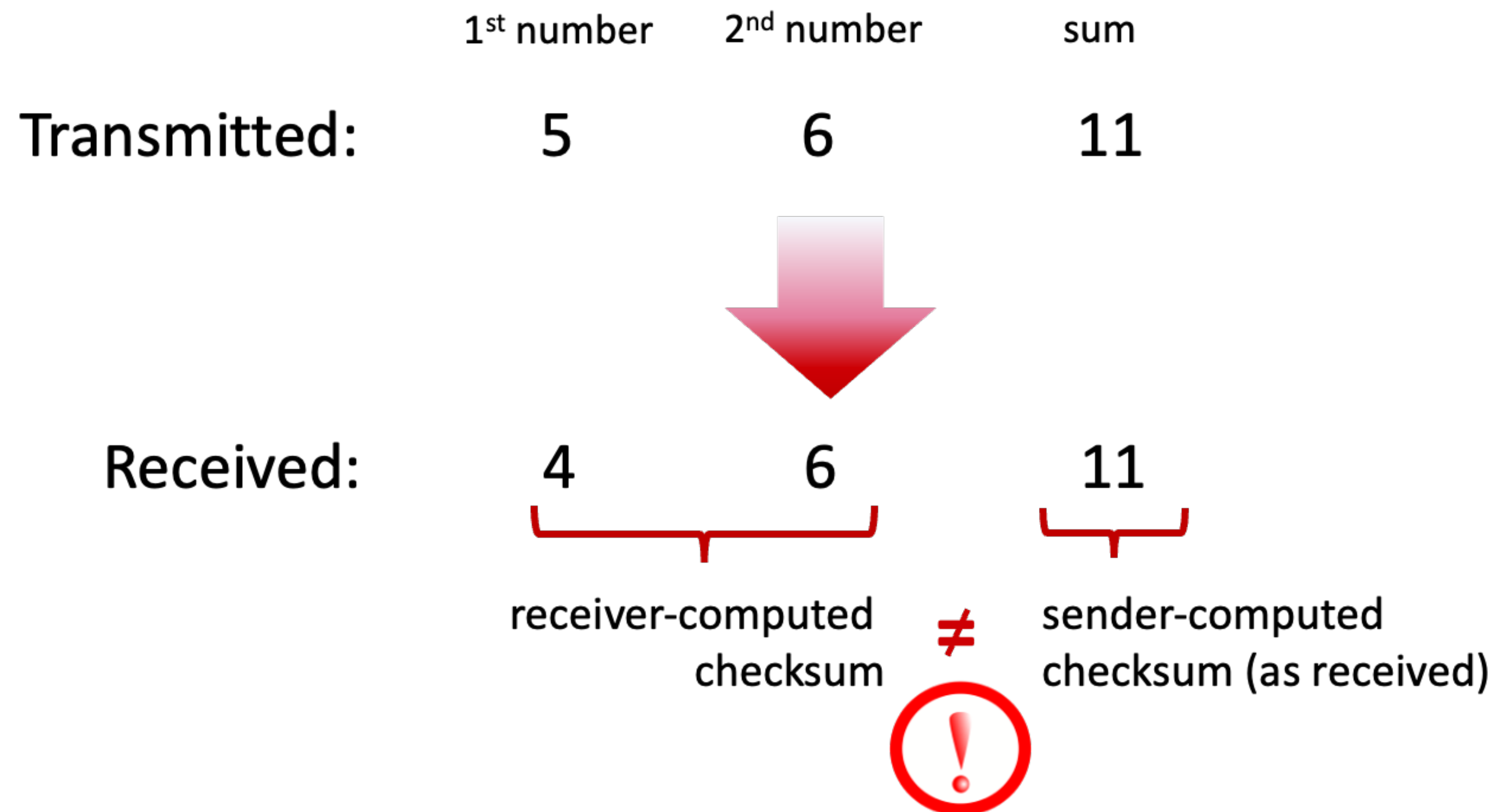
UDP segment header

- 8-byte header



UDP checksum

- Goal: detect errors (i.e., flipped bits) in the transmitted segment
- Underlying principle



UDP checksum

- Sender's operation
 - Treats content of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
 - Checksum: addition (one's complement sum) of segment content
 - Checksum value is put into the UDP checksum field
- Receiver's operation
 - Computes checksum of received segment
 - Checks if computed checksum is equal to the checksum field value
 - Not equal: error detected
 - Equal: no error detected. But maybe errors nonetheless?

Internet checksum: an example

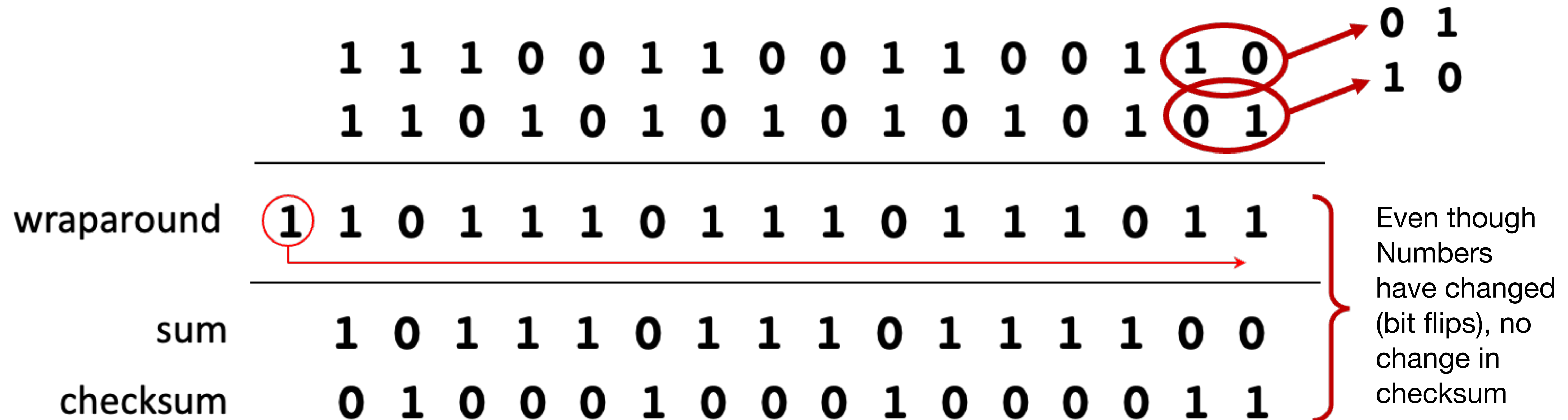
- Example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0	
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1	

* Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Internet checksum: an example

- Example: add two 16-bit integers



* Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Questions?